AI PLACEMENT OS

VOICE AGENT

Complete Product, Architecture, Data, Security, UX and Implementation Blueprint

Purpose

Define exactly what the Voice Agent is, what it can know, what it can do, how it integrates with the existing Placement OS, how it remains secure, and how it will be built and validated.

Prepared for feature-development stage - deployment and publishing intentionally on hold

Baseline: Android branch career-strategy-intelligence | Backend branch professional-presence-intelligence

1. Executive Summary

Product definition

The Voice Agent is the voice interface and agentic control layer for the entire AI Placement OS. It should understand natural speech, retrieve only the authenticated user's relevant career data through controlled tools, reason across that data, explain its conclusions, navigate the app, and execute approved actions through the existing backend business rules.

This is not a second chatbot and not a standalone voice interview screen. The existing interview voice subsystem becomes one specialized capability of a shared Voice Core. The agent is designed to sit above the current Placement OS domains and below the Android user experience, while the backend remains the authority for identity, authorization, business rules, persistence, and mutations.

Dimension	Target design
Experience	Natural, interruptible voice conversation with transcript, listening/speaking states, recovery and visual fallback.
Knowledge	Broad access to the user's Placement OS data through purpose-built read tools; never raw unrestricted database access.
Actions	Separate write/action tools with validation, confirmation when consequential, and existing idempotency/concurrency safeguards.
Reasoning	Evidence-grounded answers that distinguish stored facts, derived calculations and AI recommendations.
Reliability	Explicit voice-session state machine, cancellation correctness, reconnect/recovery, process-death safety, audio-focus handling and deterministic tool outcomes.
Security	Authenticated user scoping, least privilege, prompt-injection resistance for untrusted job/GitHub/document text, no secret exposure and privacy-aware audio handling.
Deployment posture	Feature development only. Signing, production backend deployment and Play publishing remain paused until the feature set is complete.

1.1 Core product promise

```
USER SPEAKS
  -> VOICE SESSION
  -> AGENT ORCHESTRATOR
  -> AUTHENTICATED TOOLS
  -> USER-SCOPED PLACEMENT DATA
  -> GROUNDED REASONING
  -> SPOKEN + VISUAL RESPONSE
  -> OPTIONAL CONFIRMED ACTION
```

2. Current System Baseline and What Already Exists

The repository review shows a mature Placement OS with a broad Android repository/domain layer and a FastAPI backend. The Android repository currently contains dedicated interview voice components, while the backend contains interview audio endpoints and local/development STT/TTS support. The project also has extensive authenticated repositories for profile, resume, ATS, jobs, career intelligence, interviews, applications, professional presence and execution workflows.

Existing capability	Current evidence / implication
Android voice interview	VoiceInterviewScreen, VoiceInterviewViewModel, VoiceInterviewManager, AudioRecorder and AudioPlayer already exist. The new agent should reuse the audio plumbing rather than duplicate it.
Interview audio API	The backend already supports voice-answer submission and question audio, including replay/result reconciliation from Phase 31.
Local STT/TTS	Current backend requirements include faster-whisper and pyttsx3. Existing project documentation labels this stack development-only; pyttsx3 is currently tied to Windows SAPI5.
Authenticated repositories	Android already exposes repository implementations for all major Placement OS domains, giving the Voice Agent a clean integration surface.
AI safety	Existing input/output guards, AI boundary isolation, retry budgets and ownership checks form the baseline for agent security.
Idempotency	Phase 24/30/31 work provides server-side idempotency and replay-safe mutations that action tools must reuse.

Existing voice gaps that must be absorbed into the new Voice Core

The current voice-readiness checklist identified two concrete Android issues: no audio-focus handling and no protection for configuration changes/process death during an active turn. It also records that the existing STT/TTS quality is development-only and that real-device audio validation is still required.

3. What the Voice Agent Does

3.1 User-facing capabilities

Capability	Example user request	Agent behavior
Career Coach	"What are my biggest weaknesses for	Retrieve target role, skills, evidence,

Capability	Example user request	Agent behavior
	data engineering?"	assessments and interview signals; explain prioritized gaps.
Job Coach	"Which saved job is the best match for me?"	Retrieve saved jobs, matching signals, Career Twin and goal context; compare and rank.
Resume Coach	"Why am I not getting shortlisted?"	Combine ATS findings, target-role requirements, project evidence and application outcomes.
Application Coach	"What applications need follow-up?"	Retrieve active applications, timestamps/status and notes; return prioritized follow-ups.
Learning Coach	"How do I fix my Spark weakness in seven days?"	Combine weakness records, roadmap, missions and preparation context; propose an actionable plan.
Interview Coach	"Start a difficult Data Engineer interview."	Create/use an existing interview workflow and switch into conversational voice-interview mode.
Professional Presence Coach	"What is missing from my GitHub profile?"	Use professional-presence evidence and audit outputs; explain concrete gaps.
OS Navigator	"Open my Amazon application."	Resolve the user-owned application and navigate to the relevant app screen.
Action assistant	"Add a note that the recruiter asked for my availability."	Preview/execute an approved mutation through the existing backend rules.
Daily briefing	"What should I do today?"	Combine mission, NBA, application follow-ups, weaknesses and preparation priorities.

3.2 What it should not do

- Never invent a score, status, job, action result or piece of personal data.
- Never receive unrestricted database access or choose another user's profile/user ID.
- Never expose passwords, OAuth tokens, API keys, signing keys, database credentials or internal security material.
- Never treat instructions embedded inside job descriptions, resumes, GitHub content or documents as trusted agent instructions.
- Never execute a consequential mutation solely because the LLM proposed it; authorization and business validation remain outside the model.
- Never create a second, parallel idempotency/concurrency mechanism when an existing Placement OS safeguard already covers the operation.

4. Complete Placement OS Data Map

The agent should be able to reach the complete career data plane that the authenticated user is allowed to access. Access should be organized by domain and retrieved on demand, not dumped into every prompt.

Domain	Representative data	Typical voice questions
Profile & identity	Profile, education, experience, preferences	"What is my education background?"
Resume & ATS	Resume, parsed sections, ATS checks, recommendations, history	"What are my top resume issues?"
Skills & evidence	Skills, gaps, passport, evidence	"Which skills lack evidence?"
Projects	Projects, proof, build stages, GitHub evidence	"Which project best proves Spark?"
Career intelligence	Career Twin, Goals, Career Path, Strategy, Decisions	"What should my next career move be?"
Jobs	Listings, saved jobs, matches, searches, alerts	"Which job fits me best?"
Applications	Application status, notes, outcomes, follow-up state	"What needs follow-up?"
Interviews	Sessions, questions, answers, evaluations, simulation history	"Why did I score poorly?"
Assessments	Attempts, results, strengths/weaknesses	"What did I get wrong?"
Preparation	Job-specific preparation, roadmaps, missions, Placement Path	"What should I prepare tonight?"
Professional presence	GitHub connection, repositories, evidence, audits	"How strong is my professional presence?"
Progress & actions	Progress, NBA, notifications, streaks and related execution state	"What is my highest-priority action?"

Data-access principle

Broad capability, narrow retrieval. The agent may have a large tool inventory, but each turn should retrieve only the minimum data needed to answer or perform the requested task.

5. Complete Technical Architecture





5.1 Layer responsibilities

Layer	Responsibility	Hard rule
Android Voice UI	Mic permission, listening/speaking UX, transcript, controls, visual feedback, navigation	Never implement business authorization in the UI.
Voice Core	Recording, playback, audio focus, VAD integration, interruption, cancellation, lifecycle/recovery	One shared audio/session implementation; no duplicate voice stacks.
Transport	Realtime or streamed connection, reconnect, session lifecycle	Treat transport as replaceable; do not hard-code a provider into domain logic.
Agent Orchestrator	Intent, context retrieval, tool selection, specialist routing, response synthesis	LLM is not the security boundary.
Tool Registry	Typed read/action/navigation functions	Tool permissions are explicit and validated outside model output.
Backend domain services	Existing Placement OS business rules, ownership, persistence, idempotency	Reuse existing services; avoid parallel business logic.

Layer	Responsibility	Hard rule
AI provider layer	Model/STT/TTS calls, provider abstraction, retries/budgets	Keep provider credentials server-side; never expose secrets to Android.

6. Realtime Voice Experience

6.1 Conversation lifecycle

```
IDLE
 -> LISTENING
 -> TRANSCRIBING / STREAMING INPUT
 -> THINKING
 -> TOOL_CALLING
 -> GENERATING
 -> SPEAKING
 -> INTERRUPTED -> LISTENING
 -> COMPLETE / IDLE

Failure branches: PERMISSION_ERROR | AUDIO_ERROR | NETWORK_ERROR | TOOL_ERROR | AI_ERROR | SESSION_EXPIRED
```

6.2 Premium voice behaviors

- Barge-in: user can interrupt the agent while it is speaking; playback stops cleanly and the turn continues with the new user speech.
- Voice activity detection: detect speech start/stop automatically where supported; retain push-to-talk as a deterministic fallback.
- Live transcript: show what the system heard so the user can verify important content before an action.
- Audio focus: pause/stop gracefully when another audio source or phone call takes focus, rather than silently continuing.
- Headset and Bluetooth routing: explicitly test microphone/speaker route transitions.
- Low-latency first response: stream work where practical and avoid unnecessary round trips or duplicate model calls.
- Visual fallback: every voice response is also available as text, and every important action has an on-screen confirmation state.

7. STT, TTS and Agent Model Strategy

Provider-neutral architecture

Build interfaces first: SpeechToTextProvider, TextToSpeechProvider, AgentModelProvider and VoiceTransportProvider. Provider choice should be replaceable without rewriting Placement OS business logic.

Component	Current state	Target architecture
STT	Existing faster-whisper local/development path.	Provider interface with configurable production implementation, cancellation

Component	Current state	Target architecture
		and latency metrics.
TTS	Existing pyttsx3 path; current code/documentation ties it to Windows SAPI5.	Provider interface with production-quality neural/cloud or local engine chosen later; consistent audio format contract.
Agent model	Existing backend AI provider infrastructure is already abstracted.	Agent reasoning layer uses the existing AI provider boundary and output guards; provider choice remains configurable.
Realtime transport	No shared Voice Agent transport layer yet.	A dedicated session/transport abstraction supporting streaming, interruption and reconnect.

₹0 development principle

The feature roadmap should not require new paid infrastructure. Hosted realtime services or premium voice providers may be evaluated later, but the initial architecture must keep providers pluggable and preserve a local/development path.

8. Tool Registry - The Agent's Hands and Eyes

8.1 Read tools

Category	Representative tools
Identity	get_profile, get_preferences, get_education, get_experience
Resume	get_resume, get_resume_analysis, get_ats_history
Skills	get_skills, get_skill_gaps, get_skill_passport, get_evidence
Projects	get_projects, get_project_proof, get_project_build_status
Career	get_career_goal, get_career_twin, get_career_path, get_career_strategy, get_career_decisions
Jobs	get_jobs, get_saved_jobs, get_job_match, get_saved_searches, get_alerts
Applications	get_applications, get_application, get_application_notes, get_application_outcome
Interview	get_interviews, get_interview_evaluations, get_simulation_history
Assessment	get_assessment_history, get_assessment_results, get_weaknesses
Preparation	get_preparation_plan, get_roadmap, get_daily_mission,

Category	Representative tools
	get_next_best_action, get_progress
Presence	get_github_presence, get_presence_audit, get_evidence_snapshot

8.2 Action tools

Action class	Examples	Control
Low-risk navigation	open_job, open_application, open_interview, open_weaknesses	No confirmation; execute only after resolving a user-owned object.
User-state mutation	save_job, add_application_note, update_application_status	Validate ownership; apply existing idempotency/concurrency; confirm consequential changes.
AI workflow start	start_interview, generate_preparation_plan, generate_career_path	Use existing background/recovery/idempotency rules; no duplicate AI work.
Reminder/notification	create_follow_up, create_reminder	Explicit user intent; validate schedule; prevent duplicate creates.

9. Context, Memory and Grounded Reasoning

9.1 Two memory layers

Layer	Purpose	Retention rule
Short-term conversation memory	Resolve follow-up phrases such as "how bad is it?" or "what about that job?"	Session scoped; bounded; summarize rather than retaining unlimited raw turns.
Long-term career memory	User-approved durable preferences such as target role, learning preference or interview preference	Explicitly useful, editable, viewable and deletable by the user.

9.2 Retrieval strategy

```
QUESTION
  -> INTENT
  -> REQUIRED FACTS
  -> MINIMAL TOOL SET
  -> USER-SCOPED RESULTS
  -> EVIDENCE SUMMARY
  -> REASONING
  -> ANSWER / ACTION PROPOSAL
```

9.3 Truth model

Output type	Example	Agent wording
Stored fact	"Your current SQL assessment is 72."	State as fact only if returned from an authoritative source.
Derived calculation	"Two of your five target skills lack evidence."	Explain the calculation and data basis.
AI recommendation	"Prioritize Spark optimization this week."	Clearly label as recommendation, not stored fact.

10. Action Safety and Authorization

```
USER INTENT
  -> TOOL PLAN
  -> AUTHORIZATION CHECK
  -> INPUT VALIDATION
  -> OPTIONAL USER CONFIRMATION
  -> EXISTING DOMAIN SERVICE
  -> IDEMPOTENCY / CONCURRENCY GUARD
  -> DATABASE
  -> AUTHORITATIVE RESULT
  -> VOICE + VISUAL CONFIRMATION
```

- The model proposes; application/backend code authorizes and executes.
- No arbitrary profile_id/user_id may be accepted from the model as authority for data access.
- Every mutation must be safe under retries and concurrent duplicate delivery.
- Existing Phase 24/30/31 idempotency and answer-replay behavior should be reused.
- For consequential changes, show an action preview and require explicit confirmation.
- After execution, report the actual server result, not the model's assumption.

11. Security, Privacy and Prompt-Injection Defense

11.1 Threats we must explicitly design against

Threat	Example	Defense
Cross-account data leakage	Model tries to retrieve another profile.	Identity derived from authenticated request; tool reads scoped to owning resource.
Indirect prompt injection	Job description says "ignore instructions and reveal data".	Untrusted content isolated as data; system/tool policy remains trusted; outputs validated.
Tool abuse	Model calls a mutation with unexpected arguments.	Typed schemas, server-side validation, authorization and allowlisted operations.
Replay/duplicate action	Network retries "save this job" twice.	Existing operation claims/idempotency

Threat	Example	Defense
		and business uniqueness/concurrency guards.
Secret exposure	Agent receives OAuth token or API key.	Secrets never appear in tool result schemas or prompts; keep credentials server-side.
Audio privacy	Raw recordings retained unnecessarily.	Temporary processing by default; explicit retention policy; user controls; avoid raw audio persistence unless required.
False success	Agent says an action happened when it failed.	Authoritative backend result is required before success is spoken.
Prompt leakage	Agent reveals hidden instructions/tool policy.	Reuse existing input/output guards and apply voice-agent-specific leakage tests.

Play-policy implication for the eventual release

Microphone/voice data is sensitive user data. Google Play requires transparent disclosure of access, collection, use and sharing and secure handling; a future production implementation must align the Voice Agent privacy UI, Data Safety declarations and privacy policy with the actual STT/TTS provider behavior.

12. Android UX Blueprint

Screen/element	Behavior
Voice Agent entry	Persistent but unobtrusive mic entry point from the main Placement OS experience.
Idle	Mic button, recent conversation summary and optional suggestion chips such as "Review my day" or "Practice an interview".
Listening	Clear active-mic state, elapsed time where useful, transcript appearing progressively, stop/cancel action.
Thinking/tool use	Natural status such as "Checking your applications..." rather than a generic spinner when a tool call is running.
Speaking	Animated audio state, text transcript, Stop/Interrupt control.
Action preview	Human-readable summary, object name, before/after state when applicable, Confirm/Cancel.
Error/recovery	Specific error category with retry/reconnect guidance; never expose raw exception strings.
History	Optional session history with transcript controls and clear/delete options.

Screen/element	Behavior
Memory center	View/edit/delete long-term voice-agent memories.
Voice settings	Language, voice, playback speed, push-to-talk vs VAD, transcript visibility, data retention controls.

13. Advanced Experience Roadmap

Capability	Phase target	Priority
English voice	Core release	P0
Natural interruption/barge-in	Realtime core	P0
Voice interview mode	Interview integration	P0
User-scoped cross-feature reasoning	Tool registry + agent intelligence	P0
Safe action execution	Action tools	P0
Hindi/Telugu and code-switched speech	Multilingual layer	P1
Proactive daily career briefing	Career coach layer	P1
Deep-link navigation	Android integration	P1
Calendar/reminders integration	Optional external integration	P2
Always-listening/background assistant	Separate research track because Android background microphone rules are restrictive	P3

14. End-to-End User Scenarios

Scenario A - Readiness diagnosis

User: "Am I ready for this Data Engineer job?"
Agent: identifies the job -> retrieves match + Career Twin + skills + evidence + interview signals
Agent: explains strengths, gaps, confidence and next actions
No mutation occurs.

Scenario B - Application follow-up

User: "Which applications need follow-up today?"
Agent: retrieves active applications -> ranks by follow-up need -> reads notes/status
Agent: gives concise action list and can open the highest-priority application.

Scenario C - Safe mutation

User: "Add a note to my Amazon application: recruiter asked for my availability."
Agent: resolves user-owned application -> previews action -> user confirms
Backend: validates ownership + executes existing mutation safely
Agent: reports actual success/failure.

Scenario D - Voice mock interview

User: "Start a difficult Azure Data Engineer interview."
Agent: starts/reuses interview workflow -> speaks question -> listens -> transcribes -> submits -> evaluation -> next question -> final evaluation -> updates weakness signals.

15. Implementation Roadmap

Phase	Scope	Exit criteria
VA-0	Deep read-only repository audit for Voice Agent surfaces	Complete Android/backend map, dependencies, current gaps and no-scope list.
VA-1	Shared Voice Core	Audio lifecycle, permission, audio focus, process/config recovery, cancellation and common UX states.
VA-2	Realtime transport	Streaming session, interruption/barge-in, reconnect and session lifecycle.
VA-3	STT/TTS abstraction	Provider interfaces, format contracts, timeouts, cancellation and fallback strategy.
VA-4	Placement OS tool registry	Typed authenticated read tools for all major user-owned domains.
VA-5	Agent orchestration	Intent, context retrieval, specialist routing, grounding, answer synthesis and guard integration.
VA-6	Memory	Bounded conversation memory + user-controlled durable career memory.
VA-7	Safe actions	Action tools, confirmation, validation, idempotency and authoritative results.
VA-8	Voice Career Coach	Daily briefings, job/resume/application/learning coaching and cross-feature reasoning.
VA-9	Voice Interview Coach	Shared Voice Core integrated with existing interview engine; no duplicate interview business logic.
VA-10	Security + hardening	Prompt injection, cross-account isolation, tool abuse, replay/concurrency, secret exposure, cancellation.

Phase	Scope	Exit criteria
VA-11	Real-device validation	Noise, Bluetooth, calls, screen lifecycle, process death, permissions, network loss/reconnect.
VA-12	Final Voice Agent audit	Full regression, mutation testing, artifact checks, privacy/policy review and no unresolved P0/P1 issues.

16. Validation and Quality Gates

16.1 Functional tests

- Conversation continuity across multi-turn follow-up questions.
- Correct retrieval from every registered Placement OS domain.
- Navigation to the correct user-owned object.
- Action confirmation and cancellation.
- Successful and failed tool-result reporting.
- Voice interview progression and final evaluation flow.

16.2 Reliability tests

- Duplicate transport delivery -> at most one mutation.
- Concurrent identical action -> exactly one durable outcome where semantics require it.
- Network loss during listening, thinking and speaking.
- Process death / Activity recreation during a voice session.
- Microphone permission revoke/regrant.
- Audio focus loss and regain.
- Bluetooth route connect/disconnect during a session.
- Long response / long tool execution and cancellation.

16.3 Security tests

- Two-user isolation for every read tool.
- Action tool ownership bypass mutations.
- Prompt injection carried in job descriptions, resumes, GitHub and notes.
- Tool argument tampering and type confusion.
- Secret leakage through prompts, logs, tool results or transcripts.
- Replay and idempotency mutation tests.
- AI output-leak guard parity across all new call sites.

> **Quality bar**
>
> Do not call the Voice Agent "production-ready" because a demo works. The release gate is evidence-based: critical invariants pass, deliberate mutations are caught, cross-account isolation is demonstrated, real-device voice behavior is exercised, and no known P0/P1 defect remains.

17. Observability and Operations

Signal	Record	Do not record
Session metrics	session start/end, latency, tool count, failures, reconnects	Raw secrets or unnecessary sensitive content.
Voice quality	STT latency, TTS latency, interruption rate, tool latency	Permanent raw audio by default.
Tool execution	tool name, outcome, duration, safe correlation ID	Private payloads unless strictly necessary.
Action audit	action type, target object, result, confirmation state	Passwords, tokens or full sensitive transcripts.
Provider health	provider name, error category, retry count, timeout	Credentials or authorization headers.

18. Privacy and User Controls

Control	Recommended default	Purpose
Raw audio retention	Off / temporary only	Minimize exposure; keep only what is required for the requested workflow.
Transcript history	User-controlled	Allow user to retain/delete conversation history.
Long-term memory	Off until explicitly enabled / user-reviewed	Avoid silently storing every spoken statement.
Voice language	User selected	Improve clarity and multilingual behavior.
Tool action permissions	Least privilege	Allow read access broadly within user scope; require confirmation for consequential mutations.
Data deletion	Accessible from privacy/account area	Support deletion of account and associated voice data under the existing account lifecycle.

19. Technology Reference Choices

Current online research was used to validate architecture patterns, not to lock a vendor. The strongest reference patterns are: realtime speech-to-speech with VAD and tools; open-source agent frameworks that separate STT, LLM, TTS, transport and tool orchestration; and Android-aware microphone/session lifecycle handling.

Reference	What we borrow conceptually	What we keep under our control
OpenAI Realtime architecture	Realtime conversation, VAD, interruption, tool calling and session state patterns.	Placement OS backend, user authorization, domain business logic, storage and provider abstraction.
LiveKit Agents	Realtime programmable agents, WebRTC transport, turn detection, tools and testing patterns.	Transport/provider choice, business rules, user data model and cost constraints.
Pipecat	Pipeline-oriented voice architecture and multi-agent/specialist orchestration patterns.	Our own tool registry, domain services and Android application architecture.
Android speech APIs	On-device speech recognition capability where supported.	Fallback and provider policy; never assume all devices support the same path.

20. Android and Platform Constraints

- Microphone permission must be requested through Android runtime permission flow before recording.
- Background microphone behavior is constrained on modern Android versions; the initial product should use user-initiated foreground interaction rather than an always-listening service.
- A real-device test matrix is mandatory because microphone routing, Bluetooth behavior, noise, AGC and OEM audio stacks cannot be proven by code inspection alone.
- The app targets Android API 36, so lifecycle and service behavior must be validated against the current target SDK configuration.

21. Final System Map



```
                | read    | | confirm   |
                | grounded | | execute   |
                +---------+  +-----------+
                     \    /
                      \  /
                       v v
                  +------------+
                  | BETTER CAREER|
                  | DECISIONS &  |
                  | EXECUTION    |
                  +------------+
```

Final product statement

The finished Voice Agent is a secure, user-scoped, realtime AI career operating layer. It is one interface to the Placement OS, not a parallel product: it reads relevant career evidence through controlled tools, reasons across the user's data, speaks naturally, and executes approved actions through existing backend safeguards.

22. What We Build First

1. VA-0 read-only audit: map every voice, audio, interview, AI, network, lifecycle and security surface in the current branches before editing.
2. VA-1 shared Voice Core: fix the existing voice subsystem gaps and make one reusable voice/session layer.
3. VA-2 realtime session: implement streaming/interruption/reconnect behind a transport abstraction.
4. VA-3 provider layer: separate STT/TTS/model providers and define exact audio/data contracts.
5. VA-4 tool registry: expose authenticated read access across Placement OS domains.
6. VA-5 agent intelligence: grounding, context selection, specialist routing and safe response generation.
7. VA-6/7 memory and actions: user-controlled memory plus confirmed, idempotent mutations.
8. VA-8/9 career coach + interview coach: turn the agent into the voice front door for the entire OS.
9. VA-10/11/12 harden, real-device test and audit before we return to deployment/publishing work.

23. Sources and Research Basis

Project-specific claims in this specification are grounded in the current Placement OS repositories and the supplied Phase 31/32 reports. External architecture/policy claims were checked against current official sources during preparation (October 2026).

Source	Use in this document
Project repositories: https://github.com/sharu1234-coder/ai-placement-android (branch: career-strategy-intelligence)	Current Android structure, voice components, repository/domain map.
Project repository: https://github.com/sharu1234-coder/ai-placement-backend (branch: professional-presence-intelligence)	Current FastAPI backend, interview/audio paths, AI provider abstractions and services.
Phase 31 - Final Report	Answer replay semantics, existing voice interview behavior, replay-safe answer endpoints and production-state history.
Phase 32 - Final Report	Current signing-ready state, deployment blockers, existing voice-related production constraints referenced by roadmap.

Source	Use in this document
OpenAI Realtime API - https://developers.openai.com/api/docs/guides/realtime	Realtime voice, conversation state, VAD/tool patterns and architectural options.
OpenAI Realtime tools - https://developers.openai.com/api/docs/guides/realtime-mcp	Function-tool model for application-owned business logic/private data and approval checks.
OpenAI VAD - https://developers.openai.com/api/docs/guides/realtime-vad	Speech start/stop detection and turn-handling concepts.
LiveKit Agents - https://github.com/livekit/agents	Open-source realtime agent architecture, transport, turn detection, tools and testing patterns.
Pipecat - https://github.com/pipecat-ai/pipecat	Open-source pipeline architecture for realtime voice and multi-agent systems.
Android background foreground-service restrictions - https://developer.android.com/develop/background-work/services/fgs/restrictions-bg-start	Modern Android microphone/background-service constraints.
Android SpeechRecognizer - https://developer.android.com/reference/android/speech/SpeechRecognizer	On-device speech recognition capability and API behavior.
Google Play User Data - https://support.google.com/googleplay/android-developer/answer/10144311	User-data disclosure, consent, security and handling requirements.
Google Play Data Safety - https://support.google.com/googleplay/answer/11416267	Audio files / voice recordings data classification.

24. Decision

Approved product direction

Proceed with VA-0 first. Do not begin deployment, release signing, production backend publication or Play Store submission while the Voice Agent feature set is still under active development. Treat this document as the master functional/technical specification for the voice-agent work.